Anoma Research Topics | TECHNICAL REPORT

Mailbox Actors

Jonathan Prieto-Cubides[a], **Anthony Hart**[†,a], **and Tobias Heindel**[a]

[a]Heliax AG

[†] These authors contributed equally

[*] *E-Mail: jonathan@heliax.dev*

Abstract

We present a formal framework for actor systems in which mailboxes are promoted to independent, first-class actors—native actors with their own configuration, behaviour, and lifecycle. In traditional actor models, mailboxes are opaque runtime primitives or passive data structures, limiting formal reasoning about message delivery policies. Our design decouples message storage from processing logic by automatically pairing each processing actor with a dedicated mailbox actor whose guarded-action behaviour governs delivery, filtering, and scheduling. This separation enables modular, replaceable mailbox implementations and independent scaling of message ingestion and processing, while adding no configuration burden for simple engines that need only the default first-in-first-out policy.

We formalise the system using a predicative Martin-Löf type theory with dependent types. All type families are indexed by a finite type of engine type indices, ensuring type-safe communication by construction. We provide a complete operational semantics as a labelled transition system, covering engine creation with automatic mailbox pairing, message passing, behaviour evaluation, effect execution, and engine lifecycle management. We establish type preservation, progress, effect determinism under non-overlapping guards, and mailbox isolation as metatheoretic properties.

Keywords: Actor Model ; Message-Passing Systems ; Mailbox Actors ; Operational Semantics ; Dependent Types ; Formal Methods

1. Introduction

Actor models provide a foundational framework for concurrent and distributed computation. In these models, actors are independent computational units that communicate exclusively via asynchronous message passing. The reception of messages is handled by a *mailbox*, a dedicated buffer that stores incoming messages until the actor is ready to process them [HBS73, Agh86, AMST97].

In most formalisations of the actor model, the mailbox is treated either as an opaque runtime primitive or as a passive data structure internal to the actor. Erlang [Arm10], for instance, provides mailbox interaction only through low-level system primitives with opaque handling [BHT24]. Actor-as-object models [DKVCDM16] similarly treat mailboxes as internal components with limited exposed functionality beyond basic queue operations. The Kilim framework [SM08] takes a step towards separation by decoupling

mailboxes from actors through specialised queues, but still treats them as passive data structures rather than first-class entities. Recent work on mailbox types [FAS+23] enriches the type system to reason about mailbox contents, but does not promote the mailbox itself to actor status.

This design choice has consequences for formal reasoning. When the mailbox is opaque, one cannot specify, at the level of the model, properties such as message filtering, prioritisation, or custom delivery policies. Nor can one reason modularly about the interaction between message storage and message processing. Conversely, when the mailbox is a passive data structure (as in Kilim), one gains structural separation but not behavioural independence: the mailbox cannot react to system events, enforce delivery invariants, or evolve its strategy without modifications to the actor runtime.

We propose promoting mailboxes to independent, first-class actors—native actors in the system with their own configuration, behaviour, and lifecycle. Each processing actor is automatically paired with a dedicated *mailbox actor* that manages message storage and delivery. Communication occurs through message passing to the mailbox actor, and the processing actor retrieves messages only when it is ready to handle them. This pull-based mechanism implements natural backpressure and enables flexible, replaceable mailbox implementations. Crucially, the mailbox actor is not a passive data structure: it is a fully-fledged actor with its own guarded-action behaviour, enabling custom delivery policies, message prioritisation, and filtering without modifying the processing actor or the system runtime.

We formalise this architecture in a predicative Martin-Löf type theory with dependent types. The choice of foundation is driven by the model itself: every engine component—messages, configurations, environments, behaviours—is a type family indexed by a finite type of engine type indices, so a type-safe description of engine creation and inter-engine communication requires dependent types to quantify over indices and pair them with their associated data. Alternative foundations such as set theory or rewriting logic could express the operational rules but would not enforce these typing invariants by construction; dependent types make them intrinsic, yielding definitions that are correct by construction and directly implementable in proof assistants such as Agda, Coq, or Lean. Our model introduces *engines*—a refinement of the actor notion—structured around four components: a configuration, an execution environment, a message interface, and a behaviour defined by guarded actions.

Contributions. The contributions of this paper are threefold. First, we present a dependent-type-theoretic formalisation of actor systems in which mailboxes are first-class actors (Section 3). The framework is parametric in the engine type family, making it applicable to any actor-like system follow-

ing a process-oriented paradigm. Second, we provide a complete operational semantics expressed as a labelled transition system (Section 4), covering engine creation with automatic mailbox pairing, message passing, behaviour evaluation under non-overlapping guards, effect execution, and engine lifecycle management. Third, we establish several metatheoretic properties of the system (Section 5), including type preservation under reduction, progress, effect determinism, and mailbox isolation.

Outline. Section 2 fixes notation and conventions. Section 3 introduces the engine model and its components, culminating in the definition of mailbox engines. Section 4 presents the operational semantics. Section 5 states metatheoretic properties. Section 6 discusses related work, and Section 7 concludes.

2. Notation

We work in a predicative Martin-Löf type theory with a countable hierarchy of universes

$$\mathfrak{U}_0 : \mathfrak{U}_1 : \mathfrak{U}_2 : \cdots$$

where $\mathfrak{U}_n : \mathfrak{U}_{n+1}$ for all $n \in \mathbb{N}$. The symbol $\mathfrak{U}$ denotes $\mathfrak{U}_0$ when the universe level is not important. All constructions in this paper live in $\mathfrak{U}_0$ unless stated otherwise.

Π-types. Given a type $A\colon \mathfrak{U}_n$ and a type family $B\colon A \to \mathfrak{U}_m$, the Π-type (dependent function type) $\Pi_{(x:A)}\, B(x)$ inhabits $\mathfrak{U}_{\max(n,m)}$. When B does not depend on x, we write $A \to B$.

Σ-types. Given $A\colon \mathfrak{U}_n$ and $B\colon A \to \mathfrak{U}_m$, the Σ-type (dependent pair type) $\Sigma_{(x:A)}\, B(x)$ inhabits $\mathfrak{U}_{\max(n,m)}$. Its terms are denoted $\langle a, b\rangle$ where $a\colon A$ and $b\colon B(a)$. When B does not depend on x, we write $A \times B$ and call its terms *tuples*. We generalise so that $\langle a_1, \ldots, a_n\rangle$ is a tuple of type $A_1 \times \cdots \times A_n$.

Inductive types. We use inductive type definitions in the following form:

$$\mathsf{A} := \cdots \mid \mathsf{B} \mid \cdots \mid \mathsf{C}(A_1, \ldots, A_m) \mid \cdots, \tag{1}$$

where B and C are constructors of A, with B nullary and C taking m arguments. The type A may also be indexed by parameters. Pattern matching is used for elimination.

Standard types. We denote by $\mathbb{N}$ the type of natural numbers, 0 the empty type, 1 the unit type with its unique term $*$, B the two-element type with truth values tt and ff, $\mathsf{List}(A)$ the type of lists over A, $\mathsf{Fin}(n)$ the type with exactly n elements, and $\mathsf{Maybe}(A)$ the optional type $A \oplus 1$ with constructors none and $\mathsf{some}(a)$ for $a\colon A$. The empty list is written $[]$ and $[a_1, \ldots, a_n]$ denotes a list of n elements.

Equalities and notation. We write $x := y$ for definitional equality (defining x to be y), $a \equiv b$ for judgmental equality (holding by computation), and $a = b$ for propositional equality (the identity type). In inference rules, premises use judgmental equality unless otherwise noted. A typing judgement is denoted $x : A$. We write $(_)$ to omit terms that are irrelevant or inferable from context.

Finite subsets and set-theoretic notation. Certain types in our model are equipped with decidable equality. For such types, we freely use set-theoretic notation as syntactic sugar. Finite subsets of a type A are formalised as $\mathcal{P}_{\mathsf{fin}}(A)$, implemented as lists with a uniqueness invariant. We write $a \in S$ for membership, $S \cup T$ for union, and $S \setminus T$ for difference. The disjoint union of types $A_1, \ldots, A_n$ is denoted $A_1 \oplus \cdots \oplus A_n$.

Functions. A function $f : A \to B$ applied to a is written $f(a)$. The *function update* $f[a \mapsto c]$ denotes the function agreeing with f except that input a maps to c. We overload this notation to finite sets: $S[a \mapsto c]$ replaces element a by c in S.

3. Model

This section presents a formal model of actor systems in which mailboxes are promoted to first-class actors. Building on the actor-like semantics of [HPCH25], we formalise *engines* as refined communicating Moore machines structured around four core components: a *configuration*, an *execution environment*, a *message interface*, and a *behaviour*. These components, more precisely their types, define the type signature for engines in a system, as detailed in Definition 19.

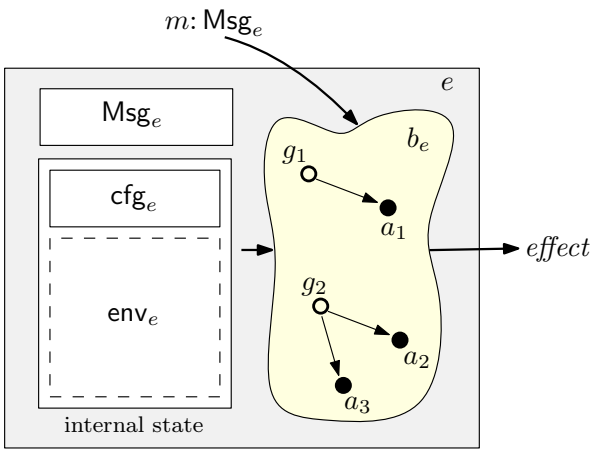

Figure 1. An engine e processing a message m conforming to its message interface Msg_e, producing an effect by evaluating its behaviour b_e in the context of its configuration cfg_e and execution environment env_e. The dashed line indicates that these components may change in a future step. The behaviour b_e consists of guarded actions $\langle g_1, a_1 \rangle$, $\langle g_2, a_2 \rangle$, and $\langle g_3, a_3 \rangle$.

The configuration conveys persistent metadata about the engine such as its parent reference and operational mode (Definition 9). The execution environment contains the engine's local state and an address book of other engines (Definition 10). The behaviour specifies how the engine reacts to messages via *guarded actions*—pairs of precondition/action that, when triggered, produce *effects*: commands requesting the system to send messages, update state, spawn new engines, or terminate (Definition 11).

For guarded action evaluation, various strategies exist for selecting which action to execute when multiple guards are satisfied. We present one such strategy based on non-overlapping guards in Section 4.6.

Remark 1 (Event-Driven Systems)**.** The behaviour of engines resembles the correct-by-construction approach used in event-driven systems such as Event-B [Dup21]. In that approach, system transitions are driven by events consisting of preconditions on the current state and actions to execute when these preconditions hold. Our guarded actions play an analogous role, with the effect system (Definition 11) restricting the available actions.

All type families in the model are indexed by a finite type of engine type indices, which we now introduce.

Definition 2 (Engine Type Index)**.** *We postulate a finite type* $\mathbb{I} : \mathfrak{U}$ *of* ***engine type indices****. For each* $i : \mathbb{I}$*, the system defines the associated type families* Msg_i*,* Cfg_i*,* Env_i*, and* EBeh_i*.*

The finiteness of $\mathbb{I}$ reflects the assumption that engine types are fixed when the system is defined. Adding support for dynamic type extension is left for future work.

3.1. Preliminaries

The model operates on a system state κ consisting of finite sets of uniquely identified nodes $\mathcal{N}$, engines $\mathcal{E}$, and in-transit messages $\mathcal{M}$. The subsequent sections detail these concepts in a bottom-up manner.

As a state machine, the system evolves in discrete steps, where the state at step i is denoted $\kappa^{(i)}$. The sets $\mathcal{N}^{(i)}$, $\mathcal{E}^{(i)}$, and $\mathcal{M}^{(i)}$ may change between steps. When no confusion arises, we omit the step index. Each engine resides on a unique node, as formalised in Definitions 19 and 22.

Remark 3 (Static Engine Allocation)**.** The model assumes that engines remain on the node where they were created. Engine migration between nodes is not considered.

Communication between engines occurs via typed messages carried by the system. A message-passing mechanism governs the delivery of in-transit messages $\mathcal{M}$, as described in Section 3.3. Each message contains sender and recipient identifiers along with a typed payload.

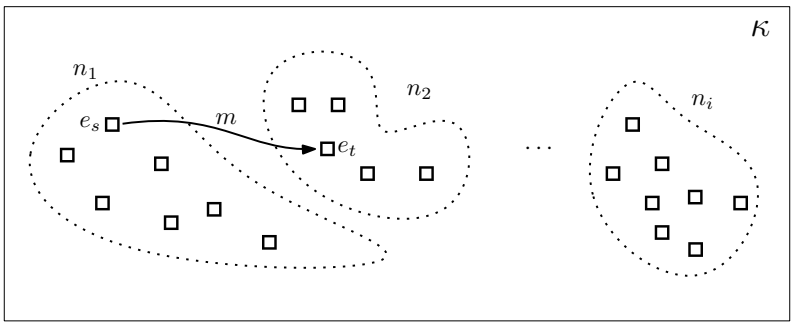

Figure 2. A system state κ with nodes $\mathcal{N}$ (dashed regions), engines $\mathcal{E}$ (boxes), and in-transit messages $\mathcal{M}$ (arrows). The message $(\langle 1, e_s\rangle \triangleright_m \langle 2, e_t\rangle)$ is in transit from engine e_s on node 1 to engine e_t on node 2.

3.2. Addresses

Definition 4 (Address)**.** *The **address** of an engine is a pair $\langle n, \iota\rangle$ of type* Addr *where n is the node identifier and ι is the engine's unique identifier within that node. Formally,*

$$\mathsf{Addr} := \mathbb{N} \times \mathbb{N}\,. \tag{2}$$

3.3. Message Structure

Engines communicate via typed messages. The type system ensures that every message payload conforms to the target engine's interface.

Definition 5 (Message Interface)**.** *For each $i : \mathbb{I}$, the **message interface** Msg_i is a type whose constructors, called **message tags**, wrap payload types that engines of type i can process. Each message tag carries a name describing its purpose, analogous to method names in object-oriented systems.*

The message interface Msg_i is presented as an inductive type in the style of (1). For a concrete system, each engine type index i fixes its message interface.

Type-safe communication is ensured by how messages are defined: there is no way to construct an ill-formed message.

Definition 6 (Message)**.** *A **message** is a triple $\langle s, t, v\rangle$ denoted $(s \triangleright_v t)$ where:*

- $s : \mathsf{Addr}$ *is the sender engine address,*
- $t : \mathsf{Addr}$ *is the target engine address, and*
- $v : \mathsf{Msg}_t$ *is the payload conforming to the target engine's message interface.*

The type of the payload v depends on the target address t: it must conform to Msg_t, the message interface of the engine at address t. This dependent typing ensures that only well-formed messages can be constructed.

To refer to the type of an arbitrary message within the system, we use the Σ-type

$$\overline{\mathsf{Msg}} := \Sigma_{(i:\,\mathbb{I})}\, \mathsf{Msg}_i\,, \tag{3}$$

which collects all valid message payloads across engine types.

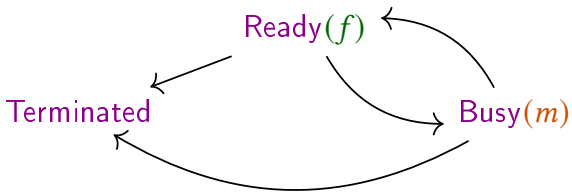

Figure 3. The engine lifecycle: an engine starts in $\mathsf{Ready}(f)$ state, where f is a predicate indicating which messages the engine accepts. Upon receiving a message m, it transitions to $\mathsf{Busy}(m)$. From there, it either returns to $\mathsf{Ready}(g)$ with a (possibly different) predicate g, or transitions to $\mathsf{Terminated}$.

3.4. Engine Components

We now define the components that constitute an engine. For each engine type index $i : \mathbb{I}$, these components are given as type families indexed by i.

Definition 7 (Engine Lifecycle)**.** *For each $i : \mathbb{I}$, the **status** of an engine of type i is a term of type $\mathsf{EStatus}_i$, determining the engine's operational state within its lifecycle.*

$$\begin{aligned} \mathsf{EStatus}_i \coloneqq\ & \mathsf{Ready}(\mathsf{Msg}_i \to \mathrm{B}) \\ \mid\ & \mathsf{Busy}(\mathsf{Msg}_i) \\ \mid\ & \mathsf{Terminated}\,. \end{aligned} \tag{4}$$

When an engine is in the $\mathsf{Ready}(f)$ state, it is available to process incoming messages. The predicate $f : \mathsf{Msg}_i \to \mathrm{B}$ acts as a filter, determining which messages the engine will accept. By default, a newly created engine starts in the $\mathsf{Ready}(\lambda x.\mathrm{tt})$ state, accepting all messages.

After creation, the engine may transition to $\mathsf{Busy}(m)$, where it is actively processing message m, or to $\mathsf{Terminated}$, where it stops processing and is subsequently removed from the system.

We distinguish between two operational modes of engines.

Definition 8 (Operational Mode)**.** *The **operational mode** of an engine is of type EMode,*

$$\mathsf{EMode} \coloneqq \mathsf{Process} \mid \mathsf{Mail}\,. \tag{5}$$

The operational mode is fixed at creation and opaque to the engine itself. The system's message-passing mechanism treats engines differently based on their mode: messages sent to $\mathsf{Process}$ engines are internally routed to their paired Mail engines, which manage message storage and delivery.

Definition 9 (Configuration)**.** *For each $i : \mathbb{I}$, we postulate a type $C_i : \mathfrak{U}$ of engine-type-specific configuration data. The **configuration** of an engine of type i is a tuple $\langle r, \mathsf{mode}, c\rangle$ of type Cfg_i, where:*

- $r : \mathsf{Maybe}(\mathsf{Addr})$ *is an optional reference to the engine's parent,*

- $\mathsf{mode}\colon \mathsf{EMode}$ *is the engine's operational mode, and*
- $c\colon C_i$ *is configuration data specific to the engine type.*

The type family of configurations is

$$\mathsf{Cfg}_i \coloneqq \mathsf{Maybe}(\mathsf{Addr}) \times \mathsf{EMode} \times C_i\,. \tag{6}$$

For an engine p of type i, we write cfg_p for its configuration value and Cfg_i for the type.

Definition 10 (Execution Environment)**.** *For each $i\colon \mathbb{I}$, we postulate a type $L_i\colon \mathfrak{U}$ for the local state. The execution* ***environment*** *of an engine of type i is a pair $\langle s, m\rangle$ of type Env_i, where:*

- $s\colon L_i$ *is the local state, and*
- $m\colon \mathsf{Name} \to \mathsf{Addr}$ *is the engine's address book, with a predefined name* SELF *for the engine's own address.*

The type family of environments is

$$\mathsf{Env}_i \coloneqq L_i \times (\mathsf{Name} \to \mathsf{Addr})\,. \tag{7}$$

For an engine p of type i, we write env_p for its environment and Env_i for the type.

3.5. Execution Directives

The behaviour of an engine specifies what it can perform when reacting to a message. How the engine reacts is determined by a set of guarded actions—pairs of *guards* (preconditions) and *actions*. When a message is processed, the system evaluates it against the engine's guards in the context of the engine's environment and configuration. If a guard is satisfied, its associated action produces an *effect*: a command to the system.

Definition 11 (Effect System)**.** *For each $i\colon \mathbb{I}$, an* ***effect*** *triggered by an engine of type i is a term of type Effect_i. Each effect represents a primitive operation that the issuing engine requests the system to execute.*

$$\begin{array}{lll}
\mathsf{Effect}_i \coloneqq & & \\
& \mathsf{Noop} & \textit{(null operation)} \\
\mid & \mathsf{Send}(\mathsf{Addr}, \mathsf{Msg}) & \textit{(message dispatch)} \\
\mid & \mathsf{Update}(\mathsf{Env}_i) & \textit{(update environment)} \\
\mid & \mathsf{Spawn}(\Sigma_{(j:\mathbb{I})}\,(\mathsf{Cfg}_j \times \mathsf{Env}_j)) & \textit{(spawn child engine of type } j\textit{)} \\
\mid & \mathsf{MFilter}(\mathsf{Msg}_i \to \mathrm{B}) & \textit{(set mailbox filter)} \\
\mid & \mathsf{Terminate} & \textit{(engine shutdown)} \\
\mid & \mathsf{Chain}(\mathsf{Effect}_i, \mathsf{Effect}_i) & \textit{(effect sequencing)}\ .
\end{array} \tag{8}$$

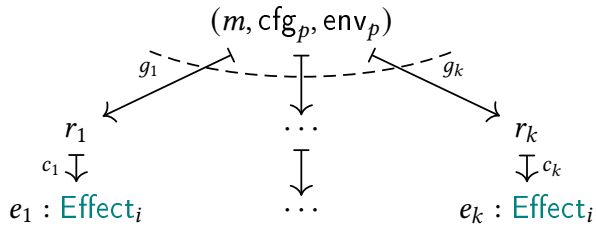

Figure 4. For an engine p of type i, its behaviour acts on a message m of type Msg_i, configuration cfg_p, and environment env_p. The guards g_k produce optional values r_k that feed into actions c_k, which return effects of type Effect_i.

Effects serve as the declarative interface specifying what an engine can do, without exposing how the behaviour computes its response.

Definition 12 (Guard)**.** *Given a type R, a **guard** for an engine of type i is a function of type $\mathsf{In}_i \to \mathsf{Maybe}(R)$ where*

$$\mathsf{In}_i \coloneqq \mathsf{Msg}_i \times \mathsf{Cfg}_i \times \mathsf{Env}_i \,. \tag{9}$$

Applied to a message, configuration, and environment, a guard returns $\mathsf{some}(r)$ *if satisfied, and* none *otherwise.*

Definition 13 (Action)**.** *An **action** for an engine of type i acting on data of type R is a function of type*

$$R \times \mathsf{In}_i \to \mathsf{Out}_i \text{ where } \mathsf{Out}_i \coloneqq \mathsf{Effect}_i \,. \tag{10}$$

Together, a guard and an action induce a function of type $\mathsf{In}_i \to \mathsf{Out}_i$.

Definition 14 (Guarded Action)**.** *Given a guard g_i and an action a_i for an engine of type i, the **guarded action** $g_i \odot a_i$ is the function of type $\mathsf{In}_i \to \mathsf{Out}_i$ defined by*

$$\begin{aligned}(g_i \odot a_i) \coloneqq \lambda\langle m, c, s\rangle.\mathsf{case}\ g_i(m)\ \mathsf{of}\ \{ \\ \mathsf{some}(r) \Rightarrow a_i(r, \langle m, c, s\rangle); \\ \mathsf{none} \Rightarrow \mathsf{Noop}\} \,.\end{aligned} \tag{11}$$

Definition 15 (Engine Behaviour)**.** *A **behaviour** for an engine of type i is a list of guarded actions, of type EBeh_i, where*

$$\mathsf{EBeh}_i \coloneqq \mathsf{List}(\mathsf{In}_i \to \mathsf{Out}_i) \,. \tag{12}$$

To ensure deterministic evaluation, we require that behaviours satisfy a well-formedness condition.

Definition 16 (Non-Overlapping Guards)**.** *A behaviour $b \colon \mathsf{EBeh}_i$ has **non-overlapping guards** if for every input $x \colon \mathsf{In}_i$, at most one guarded action in b produces a non-Noop effect. That is, for all $\langle g_j, a_j\rangle, \langle g_k, a_k\rangle \in b$ with $j \neq k$, if $g_j(x) = \mathsf{some}(r)$ then $g_k(x) = \mathsf{none}$.*

Throughout this paper, we assume that all behaviours satisfy the non-overlapping guard condition. Alternative guard selection strategies—inspired, for instance, by Dijkstra's guarded command language [Dij75]—are compatible with the model but fall outside the scope of this work.

Remark 17 (Evolving Behaviours)**.** In the present model, behaviours are fixed at engine creation. One could extend the effect system with an additional constructor for changing behaviours, at the cost of additional complexity in the operational semantics.

3.6. Engines

Engines are the core computational units of the system. Each engine processes messages of a specific type in isolation and produces effects instructing the system on actions to take. Unlike actor-as-objects and actors-as-processes models [DKVCDM16], engines are characterised by their typed components and the declarative nature of their behaviour.

Definition 18 (Engine Type)**.** *For each $i : \mathbb{I}$, the **engine type** Eng_i specifies a message interface and a behaviour:*

$$\mathsf{Eng}_i \coloneqq \mathsf{Msg}_i \times \mathsf{EBeh}_i \,. \tag{13}$$

Definition 19 (Engine)**.** *An **engine** of type Eng_i is a tuple $\langle \mathsf{Msg}_i, \mathsf{EBeh}_i, o, c, s \rangle$, where:*

- $o : \mathsf{EStatus}_i$*: the engine's status,*
- $c : \mathsf{Cfg}_i$*: the engine's configuration, and*
- $s : \mathsf{Env}_i$*: the engine's execution environment.*

For clarity, as shown in Figure 1, we include the message interface and behaviour in the tuple representation, highlighting the engine's type.

3.7. Mailbox Engines

As established in Definition 8, the system includes two kinds of engines: processing engines (mode Process) and mailbox engines (mode Mail). A mailbox engine is itself an engine—a native actor in the system—with its own configuration, environment, and behaviour. The system automatically creates a mailbox engine when a processing engine is spawned (Section 4.2). Users cannot directly create mailbox engines, though they may provide custom configuration, environment, and behaviour for the mailbox at the moment of processing engine creation. The default mailbox operates as a first-in-first-out queue.

Definition 20 (Mailbox Engine)**.** *A* mailbox engine *m for a processing engine p of type i is a system-configured engine satisfying:*

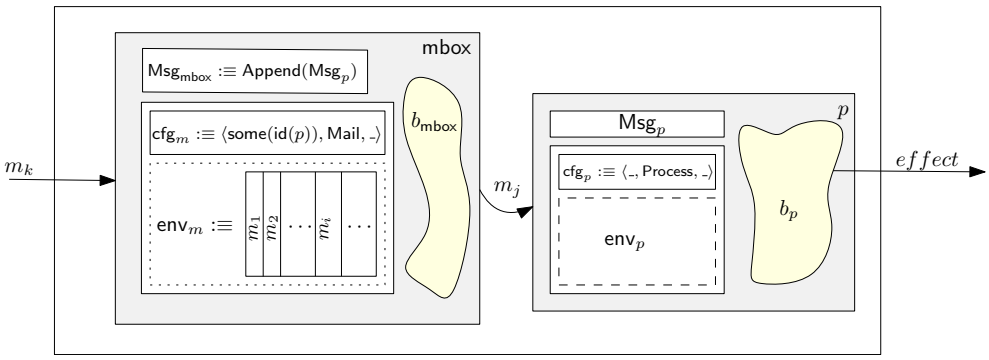

Figure 5. A processing engine p with its dedicated mailbox engine mbox. When a message m_k is sent to p, the system routes it to mbox, which stores it. When p is ready, the mailbox delivers a message determined by p's filter and the mailbox's behaviour.

- *The operational mode is* Mail.
- *The parent reference points to* p.
- *The message interface is*

$$\mathsf{Msg}_m \coloneqq \mathsf{Append}(\mathsf{Msg}_i)\,. \tag{14}$$

- *The local state of type* L_m *implements a message store with:*
  - $\emptyset$ *representing the empty state,*
  - $(\cup)\colon L_m \times \mathsf{Msg}_i \longrightarrow L_m$ *for message insertion, and*
  - $(\setminus)\colon L_m \times \mathsf{Msg}_i \longrightarrow L_m$ *for message deletion.*
- *The default behaviour* b_m *consists of a single guarded action* $\langle g_m, a_m\rangle$ *where:*
  - $g_m(\mathsf{Append}(v), c, q) \coloneqq \mathsf{some}(v)$ *: the guard always matches, extracting the payload* v *from the* Append *wrapper.*
  - $a_m(v, \mathsf{Append}(v), c, q) \coloneqq \mathsf{Update}(q \cup \{v\})$ *: the action stores the payload in the message store via an* Update *effect.*

Since the mailbox engine is a native actor, it processes incoming messages through its behaviour via s-Process, just as any other engine. The M-Enqueue rule transitions the mailbox engine from $\mathsf{Ready}(f)$ to $\mathsf{Busy}(w)$, after which the behaviour evaluates the message and produces an effect—by default, storing the payload in the message store.

As depicted in Figure 5, adding mailbox engines introduces a clear producer-consumer pattern. The separation of message storage from processing logic, opaque to sending engines, allows for independent scaling of message ingestion and core computation. The mailbox engine acts as a mediator implementing custom delivery policies via its behaviour, while the processing engine focuses on its computational tasks.

Remark 21 (Mailbox Overhead for Simple Engines)**.** For processing engines that require no custom delivery logic, one may ask whether the mailbox indirection introduces unnecessary overhead. In our design, the default mailbox is a simple first-in-first-out queue that requires no configuration from the engine implementor: it is created automatically, accepts all messages, and is transparent to sending engines. Thus the "cost" for simple engines is purely in the formalism—an additional engine instance exists—not in the user's modelling effort. The benefit is that the system uniformly supports mailbox customisation when engines evolve beyond the simple case, without requiring architectural changes.

3.8. Nodes

Definition 22 (Node)**.** *A **node** in $\mathcal{N}$ is a tuple $\langle n_i, \epsilon_i, \omega_i \rangle$ where n_i is the node's unique identifier, ϵ_i maps local identifiers to engine instances, and ω_i represents node-specific services such as unique identifier generation.*

For a node with identifier n_i, we write $\mathcal{E}_i$ for the set of engines on that node (the values of ϵ_i), and α_i for the mapping from engine instances to their addresses.

3.9. System

The system is a collection of interacting engines residing in nodes. For each $i : \mathbb{I}$, the system defines a message interface Msg_i, configuration type Cfg_i, environment type Env_i, behaviour type EBeh_i, and engine type Eng_i. These types are fixed throughout the system's lifetime and constitute the typing context.

The system structure resembles a Kripke model. We define the possible system states and then specify transition rules governing the system's evolution.

3.10. System State

Definition 23 (System State)**.** *A **system state** κ is a tuple $\langle \mathcal{N}, \mathcal{M}, \Omega \rangle$ where:*

- *$\mathcal{N}$ is the set of uniquely identified nodes,*
- *$\mathcal{M}$ is the set of in-transit messages, and*
- *Ω is the set of system-wide services.*

The superscript $\kappa^{(i)}$ denotes the system state at step i, with $\kappa^{(0)}$ the initial state. We may define $\kappa^{(0)} := \langle \emptyset, \emptyset, \emptyset \rangle$ and grow the system via the transition rules of Section 4.

The system services Ω include:

- $\mathsf{freshId}$: generates unique identifiers, and

- $\mathsf{mbox}\colon \mathsf{Addr} \to \mathsf{Addr}$: returns the address of the dedicated mailbox engine for a given processing engine.

We extend the node-local engine map ϵ_i to a global version ϵ that operates over all engines $\mathcal{E}$ in the system.

4. Operational Semantics

The operational semantics are expressed through the reduction relation $\rightsquigarrow$, where $\kappa^{(i)} \rightsquigarrow \kappa^{(i+1)}$ indicates a transition. A system state κ is represented as $(\mathcal{N}, \mathcal{M}, \Omega)$, with ϵ mapping engine addresses to instances, and α mapping instances to addresses.

4.1. Judgment Forms

The operational semantics employ five judgment forms, organised by domain.

1. **System-level operations.** $\kappa, \cdots \rightsquigarrow_{\mathsf{op}} \kappa'$ states that system operation op transforms κ into κ'.

2. **Behaviour evaluation.** $p \vdash v \downarrow E$ states that engine p processing message v produces effect E.

3. **Guarded action evaluation.** $p, \langle g, a\rangle \vdash v \downarrow E$ states that in the context of engine p, the guarded action $\langle g, a\rangle$ produces effect E when processing message v.

4. **Effect execution.** $\kappa \vdash E \rightsquigarrow \kappa'$ states that executing effect E in state κ produces state κ'.

5. **Engine processing.** $\kappa, p, m \rightsquigarrow_{\mathsf{Process}} \kappa'$ states that in state κ, engine p processing message m yields state κ'.

4.2. System-level Initialisation

Definition 24 (s-Node)**.** *A new node is created with a fresh identifier and an empty engine map.*

$$\frac{\iota := \mathsf{freshId}() \quad n \equiv \langle \iota, \emptyset, \omega_n\rangle}{\kappa \rightsquigarrow_{\mathsf{node}} (\mathcal{N} \cup \{n\}, \mathcal{M}, \Omega)}$$

4.3. Engine Creation

Engine creation involves setting up both the processing engine and its dedicated mailbox engine. The following auxiliary rule creates an individual engine; it is not a top-level system transition but is invoked by s-SpawnWithMailbox.

Definition 25 (s-EngineSpawn (auxiliary)).

$$\frac{\begin{array}{c} n_i \equiv \langle \mathsf{id}_i, \epsilon_i, \omega_i \rangle \in \mathcal{N} \\ k : \mathbb{I} \quad b : \mathsf{EBeh}_k \quad f : \mathsf{Msg}_k \to \mathrm{B} \\ \pi : \mathsf{Maybe}(\mathsf{Addr}) \quad \mathsf{cfg}_{\mathsf{new}} : \mathsf{Cfg}_k \equiv \langle \pi, o, c \rangle \\ \mathsf{env}_{\mathsf{new}} : \mathsf{Env}_k \equiv \langle s, [(\textsc{self}, \iota)] \rangle \quad \iota := \mathsf{freshId}() \quad \langle \mathsf{id}_i, \iota \rangle : \mathsf{Addr} \\ e_{\mathsf{new}} \equiv \langle \mathsf{Msg}_k, b, \mathsf{Ready}(f), \mathsf{cfg}_{\mathsf{new}}, \mathsf{env}_{\mathsf{new}} \rangle \quad \epsilon' \equiv \epsilon_i[\iota \mapsto e_{\mathsf{new}}] \end{array}}{\kappa, n_i, k, \pi, \iota, b, f, o, c, s \rightsquigarrow_{\mathsf{spawn}} (\mathcal{N}[\epsilon_i \mapsto \epsilon'], \mathcal{M}, \Omega)}$$

Spawning a processing engine requires creating its paired mailbox engine. The following rule makes this coupling explicit.

Definition 26 (s-SpawnWithMailbox). *Every processing engine is paired with a dedicated mailbox engine at creation.*

$$\frac{\begin{array}{c} k : \mathbb{I} \quad n_i \in \mathcal{N} \quad \iota_p := \mathsf{freshId}() \quad \iota_m := \mathsf{freshId}() \\ \kappa, n_i, k, \pi, \iota_p, b_k, f, \mathsf{Process}, c, s \rightsquigarrow_{\mathsf{spawn}} \kappa_1 \\ \kappa_1, n_i, k_m, \mathsf{some}(\langle \mathsf{id}_i, \iota_p \rangle), \iota_m, b_m, \lambda_.\mathsf{tt}, \mathsf{Mail}, c_m, s_m \rightsquigarrow_{\mathsf{spawn}} \kappa_2 \end{array}}{\kappa, n_i, k, \pi, b_k, f, c, s \rightsquigarrow_{\mathsf{spawn\text{-}mbox}} \kappa_2}$$

Here k_m is the engine type index for the mailbox engine paired with processing engines of type k, b_m is the default mailbox behaviour, and c_m, s_m are the initial mailbox configuration and state.

4.4. Engine Garbage Collection

To prevent zombie engine instances—engines in the Terminated state—the system removes them.

Definition 27 (s-Clean). *An engine whose status is Terminated is removed from its node's engine map.*

$$\frac{n_i \in \mathcal{N} \quad p \in \mathcal{E}_i \quad \mathsf{status}(p) \equiv \mathsf{Terminated}}{\kappa \rightsquigarrow_{\mathsf{clean}} (\mathcal{N}[\epsilon_i \mapsto \epsilon_i \setminus \{p\}], \mathcal{M}, \Omega)}$$

The s-Clean rule does not remove the engine's mailbox. The mailbox engine remains to absorb messages that may still be in transit, preventing message loss. It can be cleaned independently once it is no longer needed.

Remark 28 (Messages to Terminated Engines). When a message in transit is addressed to a processing engine that has been terminated and cleaned, the message's target address resolves to the (still existing) mailbox engine. Since the mailbox engine accepts all messages via its Append interface, the message is enqueued but never dequeued, as no processing engine exists to consume it. This design choice silently absorbs stale messages rather than introducing error-handling complexity.

4.5. Message-Passing Mechanism

The message-passing mechanism supports asynchronous communication with non-blocking sends, mailbox isolation with eventual delivery, and explicit lifecycle management.

Definition 29 (M-Send)**.** *A message with payload v conforming to the target's interface is wrapped in the* Append *constructor and placed in transit, addressed to the target engine's dedicated mailbox.*

$$\frac{e_s, e_t \in \mathcal{E} \quad \mathsf{mode}(e_t) \equiv \mathsf{Process} \quad v : \mathsf{Msg}_t \qquad \kappa \equiv (\mathcal{N}, \mathcal{M}, \Omega)}{\kappa, \alpha(e_t), v \rightsquigarrow_{\mathsf{send}} (\mathcal{N}, \mathcal{M} \cup \{(\alpha(e_s) \triangleright_{\mathsf{Append}(v)} \mathsf{mbox}(\epsilon(e_t)))\}, \Omega)}$$

The M-Send rule redirects messages to the target engine's dedicated mailbox, wrapping the payload in an Append constructor. This is the only way to add a message to $\mathcal{M}$.

Definition 30 (M-Enqueue)**.** *An in-transit message addressed to a mailbox engine is consumed from $\mathcal{M}$, transitioning the mailbox engine from* $\mathsf{Ready}(f)$ *to* $\mathsf{Busy}(w)$*. The mailbox engine then processes the message through its behaviour via* s-Process*.*

$$\frac{m \equiv (_ \triangleright_w m_t) \in \mathcal{M} \quad m_t \in \mathcal{E} \qquad m_t \equiv \langle _, _, \mathsf{Ready}(f), _, _ \rangle \quad f(w) \equiv \mathsf{tt}}{\kappa \rightsquigarrow_{\mathsf{enqueue}} (\mathcal{N}[\epsilon(m_t) \mapsto \langle _, _, \mathsf{Busy}(w), _, _ \rangle], \mathcal{M} \setminus \{m\}, \Omega)}$$

Definition 31 (M-Dequeue)**.** *When a processing engine is ready and its mailbox contains a message passing the engine's filter, the message is removed from the mailbox and the engine transitions to the busy state.*

$$\frac{p \in \mathcal{E} \quad p \equiv \langle _, _, \mathsf{Ready}(f), _, _ \rangle \qquad m_p \in \mathcal{E} \quad m_p \equiv \mathsf{mbox}(p) \quad \mathsf{env}(\epsilon(m_p)) \equiv q \cup \{v\} \quad f(v) \equiv \mathsf{tt} \qquad \mathcal{N}' \equiv \mathcal{N}[\epsilon(p) \mapsto \langle _, _, \mathsf{Busy}(v), _, _ \rangle][m_p \mapsto \langle _, _, _, _, q \rangle]}{\kappa \rightsquigarrow_{\mathsf{dequeue}} (\mathcal{N}', \mathcal{M}, \Omega)}$$

Note that the M-Dequeue rule requires the filter predicate f to accept the message v, that is, $f(v) \equiv \mathsf{tt}$. A message v in the mailbox for which $f(v) \equiv \mathsf{ff}$ remains in the mailbox until the filter is updated or the engine is terminated.

4.6. Behaviour Evaluation

This subsection formalises how an engine evaluates its guarded actions in response to a message.

Definition 32 (B-GuardedActionEval)**.** *A single guarded action is evaluated: if the guard is satisfied, the action is applied to produce an effect.*

$$\frac{p \in \mathcal{E} \quad v : \mathsf{Msg}_p \quad p \equiv \langle _, b_p, \mathsf{Busy}(v), \mathsf{cfg}_p, \mathsf{env}_p \rangle \quad \langle g, a \rangle \in b_p \quad g(v, \mathsf{cfg}_p, \mathsf{env}_p) \equiv \mathsf{some}(r)}{p, \langle g, a \rangle \vdash v \downarrow a(r, v, \mathsf{cfg}_p, \mathsf{env}_p)}$$

Definition 33 (B-NoMatchingGuard)**.** *If a guard returns* none*, the guarded action produces* Noop*.*

$$\frac{p \in \mathcal{E} \quad p \equiv \langle _, b_p, \mathsf{Busy}(_), \mathsf{cfg}_p, \mathsf{env}_p \rangle \quad \langle g, a \rangle \in b_p \quad g(v, \mathsf{cfg}_p, \mathsf{env}_p) \equiv \mathsf{none}}{p, \langle g, a \rangle \vdash v \downarrow \mathsf{Noop}}$$

Guard Selection Strategy. Under the non-overlapping guard assumption (Definition 16), at most one guarded action produces a non-Noop effect for any given input. The following rule selects the unique matching guard.

Definition 34 (B-GuardStrategy)**.**

$$\frac{p \in \mathcal{E} \quad p \equiv \langle _, b_p, \mathsf{Busy}(v), _, _ \rangle \quad \langle g_i, a_i \rangle \in b_p \quad p, \langle g_i, a_i \rangle \vdash v \downarrow E \quad \forall_{j \neq i}\, \langle g_j, a_j \rangle \in b_p \,.\, p, \langle g_j, a_j \rangle \vdash v \downarrow \mathsf{Noop}}{p \vdash v \downarrow E}$$

When no guard matches the message, the overall effect is Noop.

Definition 35 (B-AllGuardsFail)**.**

$$\frac{p \in \mathcal{E} \quad p \equiv \langle _, b_p, \mathsf{Busy}(v), _, _ \rangle \quad \forall\, \langle g_j, a_j \rangle \in b_p \,.\, p, \langle g_j, a_j \rangle \vdash v \downarrow \mathsf{Noop}}{p \vdash v \downarrow \mathsf{Noop}}$$

When b_p is the empty list, the universal quantifier in B-AllGuardsFail is vacuously satisfied, so the rule produces $p \vdash v \downarrow \mathsf{Noop}$.

4.7. Effect Execution

Given a system state κ, an effect E, and a successor state κ', the judgment $\kappa \vdash E \rightsquigarrow \kappa'$ states that executing E in κ produces κ'. This judgment is defined by case analysis on E.

Definition 36 (E-Send)**.** *Executing a* Send *effect dispatches a message via the M-Send rule.*

$$\frac{e_s, e_t \in \mathcal{E} \quad l_t \equiv \alpha(e_t) \quad v : \mathsf{Msg}_s \quad m : \mathsf{Msg}_t \quad \vdots \quad e_s \vdash v \downarrow \mathsf{Send}(l_t, m) \quad \kappa, l_t, m \rightsquigarrow_{\mathsf{send}} \kappa'}{\kappa \vdash \mathsf{Send}(l_t, m) \rightsquigarrow \kappa'}$$

Definition 37 (E-Update). *Executing an* Update *effect replaces the engine's execution environment with the provided value.*

$$\frac{p \in \mathcal{E} \quad \mathsf{env}_p, s : \mathsf{Env}_p}{(\mathcal{N}, \mathcal{M}, \Omega) \vdash \mathsf{Update}(s) \rightsquigarrow (\mathcal{N}[\epsilon(e_p) \mapsto \langle _, _, _, _, s \rangle], \mathcal{M}, \Omega)}$$

Definition 38 (E-MFilter).

$$\frac{\begin{array}{c} p \in \mathcal{E} \quad f : \mathsf{Msg}_p \to \mathrm{B} \\ \vdots \\ p \vdash v \downarrow \mathsf{MFilter}(f) \end{array}}{(\mathcal{N}, \mathcal{M}, \Omega) \vdash \mathsf{MFilter}(f) \rightsquigarrow (\mathcal{N}[\epsilon(p) \mapsto \langle _, _, \mathsf{Ready}(f), _, _ \rangle], \mathcal{M}, \Omega)}$$

Note that E-MFilter transitions the engine from $\mathsf{Busy}(v)$ *to* $\mathsf{Ready}(f)$*, setting the filter for the next message cycle. When composed via E-Chain, this transition takes priority over the default return to* $\mathsf{Ready}(\lambda_.\mathsf{tt})$ *in s-Process.*

Definition 39 (E-Spawn). *Executing a* Spawn *effect creates a child engine via s-SpawnWithMailbox, ensuring it is paired with a mailbox engine.*

$$\frac{\begin{array}{c} n_i \in \mathcal{N} \quad p \in \mathcal{E}_i \quad j : \mathbb{I} \quad \iota := \mathsf{freshId}() \\ c \equiv \langle \alpha(p), \mathsf{Process}, _ \rangle \quad s : \mathsf{Env}_j \quad b : \mathsf{EBeh}_j \\ \kappa, n_i, j, \mathsf{some}(\alpha(p)), \iota, b, \lambda_.\mathsf{tt}, \mathsf{Process}, c, s \rightsquigarrow_{\mathsf{spawn\text{-}mbox}} \kappa' \end{array}}{\kappa \vdash \mathsf{Spawn}(\langle j, c, s \rangle) \rightsquigarrow \kappa'}$$

Definition 40 (E-Chain). *Executing a* Chain *effect sequences two sub-effects: the first is executed in the current state, and the second in the resulting state.*

$$\frac{\begin{array}{c} p \in \mathcal{E} \quad v : \mathsf{Msg}_p \\ \vdots \\ p \vdash v \downarrow \mathsf{Chain}(\mathit{eff}_1, \mathit{eff}_2) \quad \kappa \vdash \mathit{eff}_1 \rightsquigarrow \kappa' \quad \kappa' \vdash \mathit{eff}_2 \rightsquigarrow \kappa'' \end{array}}{\kappa \vdash \mathsf{Chain}(\mathit{eff}_1, \mathit{eff}_2) \rightsquigarrow \kappa''}$$

Definition 41 (E-Terminate).

$$\frac{\begin{array}{c} p \in \mathcal{E} \quad v : \mathsf{Msg}_p \\ \vdots \\ p \vdash v \downarrow \mathsf{Terminate} \end{array}}{\kappa \vdash \mathsf{Terminate} \rightsquigarrow (\mathcal{N}[\epsilon(p) \mapsto \langle _, _, \mathsf{Terminated}, _, _ \rangle], \mathcal{M}, \Omega)}$$

The E-Terminate effect sets the engine's status to Terminated*. The s-Clean rule subsequently removes the engine from the system.*

Definition 42 (E-Noop). *Executing a* Noop *effect leaves the system state unchanged.*

$$\frac{p \in \mathcal{E} \quad v : \mathsf{Msg}_p \quad \vdots \quad p \vdash v \downarrow \mathsf{Noop}}{\kappa \vdash \mathsf{Noop} \rightsquigarrow \kappa}$$

4.8. Engine Processing

The s-Process rule coordinates behaviour evaluation, effect execution, and the engine's return to the ready state.

Definition 43 (s-Process).

$$\frac{\begin{array}{c} n_i \in \mathcal{N} \quad p \in \mathcal{E}_i \quad p \equiv \langle i_p, b_p, \mathsf{Busy}(m), \mathsf{cfg}_p, \mathsf{env}_p \rangle \\ p \vdash m \downarrow E \qquad \kappa \vdash E \rightsquigarrow \kappa' = (\mathcal{N}', \mathcal{M}', \Omega') \\ p' \equiv \epsilon'(\alpha(p)) \qquad o' \equiv \begin{cases} \mathsf{Ready}(\lambda_.\mathsf{tt}) & \textit{if}\ \mathsf{status}(p') = \mathsf{Busy}(_) \\ \mathsf{status}(p') & \textit{otherwise} \end{cases} \\ \epsilon'' \equiv \epsilon'[\alpha(p) \mapsto \langle i_p, b_p, o', \mathsf{cfg}_{p'}, \mathsf{env}_{p'} \rangle] \end{array}}{\kappa, p, m \rightsquigarrow_{\mathsf{Process}} (\mathcal{N}'[\epsilon' \mapsto \epsilon''], \mathcal{M}', \Omega')}$$

Let p' denote the engine at address $\alpha(p)$ after effect execution. If the effects did not change the engine's status (i.e. it remains $\mathsf{Busy}(_)$), s-Process transitions it to $\mathsf{Ready}(\lambda_.\mathsf{tt})$. If an E-MFilter or E-Terminate effect already changed the status, that status is preserved. This design ensures that filters set by MFilter take effect immediately, while providing a sensible default when no filter is specified.

4.9. System State Transition Rules

The inference rules above collectively govern system state transitions. Every system-level operation induces a state transition:

$$\kappa, \cdots \rightsquigarrow_{\mathsf{op}} \kappa' \quad \text{implies} \quad \kappa \rightsquigarrow \kappa' \tag{15}$$

for any operation op. Figure 6 summarises all rules.

Judgment form	Rules
System-level operations	s-Node, s-SpawnWithMailbox, s-Clean, M-Send, M-Enqueue, M-Dequeue
Behaviour evaluation	B-GuardStrategy, B-AllGuardsFail
Guarded action evaluation	B-GuardedActionEval, B-NoMatchingGuard
Effect execution	E-Send, E-Update, E-MFilter, E-Spawn, E-Chain, E-Terminate, E-Noop
Engine processing	s-Process

Figure 6. Rules for the operational semantics, organised by judgment form.

5. Properties

We state several metatheoretic properties of the system. Full proofs are deferred to an extended version; here we provide proof sketches.

We first define well-typedness of system states. A system state $\kappa = (\mathcal{N}, \mathcal{M}, \Omega)$ is *well-typed* if:

1. every engine $p \in \mathcal{E}$ of type Eng_i has $\mathsf{status}(p) : \mathsf{EStatus}_i$, $\mathsf{cfg}_p : \mathsf{Cfg}_i$, and $\mathsf{env}_p : \mathsf{Env}_i$;
2. every message $(s \triangleright_v t) \in \mathcal{M}$ satisfies $v : \mathsf{Msg}_t$; and
3. for every processing engine p, the function mbox returns a valid mailbox engine address.

Proposition 44 (Type Preservation)**.** *If $\kappa \rightsquigarrow \kappa'$ and κ is well-typed, then κ' is well-typed.*

Proof sketch. By case analysis on the transition rule applied. s-Node adds a node with an empty engine map, trivially preserving invariants. s-EngineSpawn creates an engine with correctly typed components; s-SpawnWithMailbox composes two such creations. For message passing: M-Send constructs messages with payloads satisfying $v : \mathsf{Msg}_t$ by premise, wrapping in Append; M-Enqueue preserves the message type when transitioning the mailbox engine to $\mathsf{Busy}(w)$, where w has type Msg_m; M-Dequeue preserves the type family when moving a message to the processing engine's busy state. For effect execution: E-Update requires $s : \mathsf{Env}_p$; E-Spawn delegates to s-SpawnWithMailbox; E-Terminate and E-MFilter change only the status, remaining in $\mathsf{EStatus}_i$; E-Chain composes two type-preserving steps; E-Noop is the identity. s-Process evaluates the behaviour, executes the resulting effect, and restores the engine to $\mathsf{Ready}(f)$ or preserves the status set by the effect. s-Clean removes an engine without affecting others. □

Proposition 45 (Progress)**.** *A well-typed system state κ in which either (i) $\mathcal{M} \neq \emptyset$, or (ii) there exists an engine in state $\mathsf{Busy}(m)$, or (iii) there exists an engine in state $\mathsf{Terminated}$, can take a step.*

Proof sketch. We consider each case. (i) If $\mathcal{M} \neq \emptyset$, let $(s \triangleright_w t) \in \mathcal{M}$. By well-typedness, t is a valid mailbox engine address. Since $\mathsf{mbox}()$ maps processing engines to their mailboxes and the message is addressed to a mailbox, the target mailbox engine exists. If the mailbox engine is in state $\mathsf{Ready}(f)$, then since f is $\lambda_.\mathsf{tt}$ by default, the premises of M-Enqueue are satisfied. If the mailbox engine is $\mathsf{Busy}(_)$, then case (ii) applies to it and s-Process fires, returning it to $\mathsf{Ready}(_)$. (ii) If an engine p is in state $\mathsf{Busy}(m)$, the behaviour b_p is a finite list of guarded actions. Each guard g_j applied to the input either

returns $\mathsf{some}(r)$ or none. If some guard matches, B-GuardStrategy applies; otherwise B-AllGuardsFail applies. In either case, a judgment $p \vdash m \downarrow E$ is derivable, and s-Process fires. (iii) If an engine p has status Terminated, then $\mathsf{status}(p) \equiv \mathsf{Terminated}$ and s-Clean applies. □

Proposition 46 (Effect Determinism)**.** *Under the non-overlapping guard assumption (Definition 16), for any engine p in state $\mathsf{Busy}(v)$, the judgment $p \vdash v \downarrow E$ determines E uniquely.*

Proof sketch. Guards are pure functions of type $\mathsf{In}_i \to \mathsf{Maybe}(R)$, so each $g_j(v, \mathsf{cfg}_p, \mathsf{env}_p)$ is determined. By the non-overlapping guard condition, for any input at most one guard g_i returns $\mathsf{some}(r)$ while all others return none. Thus: if exactly one guard matches, B-GuardStrategy selects it uniquely and the effect is $a_i(r, v, \mathsf{cfg}_p, \mathsf{env}_p)$, which is determined since actions are functions. If no guard matches, B-AllGuardsFail yields Noop. These two cases are exhaustive and mutually exclusive, so E is uniquely determined. □

Proposition 47 (Mailbox Isolation)**.** *A processing engine never directly receives a message from $\mathcal{M}$. All messages are mediated by the corresponding mailbox engine.*

Proof sketch. By inspection of the M-Send rule: its premise $\mathsf{mode}(e_t) \equiv \mathsf{Process}$ restricts the target to a processing engine, and its conclusion redirects the message to $\mathsf{mbox}(\epsilon(e_t))$. The M-Enqueue rule delivers messages only to mailbox engines by transitioning them to $\mathsf{Busy}(w)$. The M-Dequeue rule is the sole mechanism by which a processing engine transitions from $\mathsf{Ready}(f)$ to $\mathsf{Busy}(v)$, and it reads from the paired mailbox engine. Hence, no rule places a message directly in a processing engine's state. □

Proposition 48 (Eventual Delivery)**.** *Under a weak fairness assumption—every continuously applicable rule is eventually applied—if a message $m \in \mathcal{M}$ targets a non-terminated processing engine, then m is eventually enqueued in the target's mailbox.*

Proof sketch. By M-Send, the message m is addressed to the target's mailbox engine. The mailbox engine's status is independent of its parent processing engine's termination: s-Clean removes terminated engines, but the mailbox engine persists until it is independently terminated (Remark 28). Since the processing engine is non-terminated, neither is its mailbox. The mailbox engine accepts all messages (its filter is $\lambda_.\mathsf{tt}$), so M-Enqueue is applicable whenever the mailbox is in state $\mathsf{Ready}(_)$. If the mailbox is $\mathsf{Busy}(_)$, weak fairness ensures s-Process eventually fires, returning it to $\mathsf{Ready}(_)$. Thus M-Enqueue is eventually applicable and, under weak fairness, eventually applied—consuming m from $\mathcal{M}$ and transitioning the mailbox to $\mathsf{Busy}(m)$.

s-Process then evaluates the mailbox behaviour, storing the payload in the mailbox's local state. □

6. Related Work

Actor Model Foundations. The actor model, introduced by Hewitt, Bishop, and Steiger [HBS73] and developed by Agha [Agh86, AMST97], provides the conceptual basis for our work. Our engine model refines the actor concept by introducing typed components—configurations, environments, message interfaces, and guarded-action behaviours—while preserving the fundamental principles of encapsulation and asynchronous message passing.

Process Calculi. Process calculi such as CCS and the π-calculus [Mil89] provide foundational frameworks for reasoning about concurrent communication. Our system shares the emphasis on labelled transitions and compositional semantics, but differs in treating message storage as a first-class concern through mailbox engines rather than relying on channel-based communication.

Erlang. The process-oriented architecture of Erlang [Arm10, Arm97] is the closest practical counterpart to our model. Its formal semantics [BHT24] defines processes and message passing but treats the mailbox as an opaque runtime primitive. Our work makes the mailbox explicit and programmable by promoting it to actor status.

Session and Behavioural Types. Session types [HYC08] and behavioural types for actors [FHK+24, FAS+23] provide type-theoretic guarantees for communication protocols. Our dependent type families for message interfaces serve a related purpose—ensuring type-safe communication—though we focus on the operational semantics of mailbox management rather than protocol compliance. Mailbox types [FAS+23] are particularly relevant, as they enrich the type system to reason about mailbox contents, complementing our approach of making the mailbox a first-class entity.

Event-Driven Systems. The correct-by-construction approach of Event-B [Dup21] inspires our guarded-action formulation. Our guards and actions parallel Event-B events, with the effect system constraining the available operations. The connection to Dijkstra's guarded command language [Dij75] is also apparent in the non-overlapping guard assumption.

Mailbox Decoupling. The Kilim framework [SM08] achieves mailbox decoupling through specialised queues where actors pull messages when ready. However, Kilim treats mailboxes as passive data structures. Actor selector patterns [IS14] provide mechanisms for selective message reception but do not promote the mailbox itself. Our work combines Kilim's structural

separation with the programmability of first-class actors, enabling custom delivery policies, message prioritisation, and independent scaling of message ingestion.

Formalisation of Actor Systems. Talcott [Tal98] provides an actor theory in rewriting logic, and Actario [YW15] develops a Coq formalisation of the actor model. Our dependent-type-theoretic approach offers a middle ground: more structured than rewriting-based approaches, while remaining independent of a specific proof assistant. The framework of [HPCH25] provides the actor-like semantics that we build upon, extending it with mailbox engines and metatheoretic properties.

7. Conclusion

We have presented a formal framework for actor systems in which mailboxes are promoted to independent, first-class actors—native actors with their own configuration, behaviour, and lifecycle. Unlike prior approaches that treat mailboxes as opaque primitives [BHT24] or passive data structures [SM08], our mailbox actors are fully-fledged engines whose guarded-action behaviours govern delivery, filtering, and scheduling. The dependent-type-theoretic formulation, with all type families indexed by a finite type of engine type indices, ensures type-safe communication by construction.

The operational semantics cover engine creation with automatic mailbox pairing, asynchronous message passing through mailbox intermediaries, behaviour evaluation under non-overlapping guards, effect execution, and engine lifecycle management. We established type preservation, progress, effect determinism, mailbox isolation, and eventual delivery as metatheoretic properties. The mailbox isolation property, in particular, captures the key design invariant: all messages to a processing engine are mediated by its dedicated mailbox actor, which is guaranteed by the structure of the transition rules.

Limitations. The model assumes static engine types (no dynamic type extension), static engine allocation (no migration between nodes), and fixed behaviours (no runtime behaviour modification). The non-overlapping guard assumption, while ensuring determinism, restricts the expressiveness of behaviours. Multiple mailboxes per processing engine are excluded to avoid race conditions and preserve delivery guarantees.

Future Work. Several extensions are natural. First, mechanisation of the framework and its proofs in a proof assistant such as Agda or Coq would provide machine-checked guarantees. Second, a prototype implementation in Elixir—which provides a robust actor runtime through the Erlang VM—would validate the practical applicability. Third, dynamic engine type extension

would enable open systems. Finally, relaxing the non-overlapping guard assumption to permit prioritised or nondeterministic guard selection would broaden the class of expressible behaviours.

8. Acknowledgements

We would like to express our gratitude to Jeremy Ornelas and Christophe De Troyer, whose insightful discussions inspired the development of this work.

References

Agh86. Gul A. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems.* MIT Press, 1986. (cit. on pp. 1 and 21.)

AMST97. Gul A. Agha, Ian A. Mason, Scott F. Smith, and Carolyn L. Talcott. A foundation for actor computation. *J. Funct. Program.*, 7(1):1–72, January 1997. doi:10.1017/S095679689700261X. (cit. on pp. 1 and 21.)

Arm97. Joe Armstrong. The development of erlang. *SIGPLAN Not.*, 32(8):196–203, aug 1997. doi:10.1145/258949.258967. (cit. on p. 21.)

Arm10. Joe Armstrong. Erlang. *Communications of the ACM*, 53(9):68–75, 2010. (cit. on pp. 1 and 21.)

BHT24. Péter Bereczky, Dániel Horpácsi, and Simon Thompson. Program equivalence in the erlang actor model. *Computers*, 13(11), 2024. URL: https://www.mdpi.com/2073-431X/13/11/276, doi:10.3390/computers13110276. (cit. on pp. 1, 21, and 22.)

Dij75. Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Commun. ACM*, 18(8):453–457, August 1975. doi:10.1145/360933.360975. (cit. on pp. 10 and 21.)

DKVCDM16. Joeri De Koster, Tom Van Cutsem, and Wolfgang De Meuter. 43 years of actors: a taxonomy of actor models and their key properties. In *Proceedings of the 6th International Workshop on Programming Based on Actors, Agents, and Decentralized Control*, AGERE 2016, page 31–40, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/3001886.3001890. (cit. on pp. 1 and 10.)

Dup21. Guillaume Dupont. *Correct-by-Construction Design of Hybrid Systems Based on Refinement and Proof.* Theses, Institut National Polytechnique de Toulouse - INPT, January 2021. URL: https://theses.hal.science/tel-04165215. (cit. on pp. 5 and 21.)

FAS+23. Simon Fowler, Duncan Paul Attard, Franciszek Sowul, Simon J. Gay, and Phil Trinder. Special delivery: Programming with mailbox types. *Proceedings of the ACM on Programming Languages*, 7(ICFP):78–107, August 2023. URL: http://dx.doi.org/10.1145/3607832, doi:10.1145/3607832. (cit. on pp. 2 and 21.)

FHK+24. Simon Fowler, Philipp Haller, Roland Kuhn, Sam Lindley, Alceste Scalas, and Vasco T. Vasconcelos. Behavioural types for heterogeneous systems (position paper). *Electronic Proceedings in Theoretical Computer Science*, 401:37–48, April 2024. URL: http://dx.doi.org/10.4204/EPTCS.401.4, doi:10.4204/eptcs.401.4. (cit. on p. 21.)

HBS73. Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. pages 235–245. Morgan Kaufmann Publishers Inc., 1973. (cit. on pp. 1 and 21.)

HPCH25. Tobias Heindel, Jonathan Prieto-Cubides, and Anthony Hart. Dynamic Effective Timed Communication Systems. *Anoma Research Topics*, Mar 2025. URL: https://doi.org/10.5281/zenodo.14984147, doi:10.5281/zenodo.14984148. (cit. on pp. 4 and 22.)

HYC08. Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 273–284, 2008. (cit. on p. 21.)

IS14. Shams M. Imam and Vivek Sarkar. Selectors: Actors with multiple guarded mailboxes. SPLASH '14. ACM, October 2014. URL: http://dx.doi.org/10.1145/2687357.2687360, doi:10.1145/2687357.2687360. (cit. on p. 21.)

Mil89. R. Milner. *Communication and Concurrency*. Prentice-Hall, Inc., USA, 1989. (cit. on p. 21.)

SM08. Sriram Srinivasan and Alan Mycroft. Kilim: Isolation-typed actors for java. In Jan Vitek, editor, *ECOOP 2008 – Object-Oriented Programming*, pages 104–128, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. (cit. on pp. 1, 21, and 22.)

Tal98. Carolyn L. Talcott. Composable semantic models for actor theories. *Higher Order Symbolic Computation*, 11(3):281–343, 1998. URL: http://dx.doi.org/10.1023/A:1010042915896, doi:10.1023/a:1010042915896. (cit. on p. 22.)

YW15. Shohei Yasutake and Takuo Watanabe. Actario: A framework for reasoning about actor systems. 2015. (cit. on p. 22.)